CREDIT RISK MODELLING

A Probability-of-Default Scorecard

Sajal Agarwal - November 2026

WHAT THIS PROJECT IS, IN ONE LINE

A single logistic-regression scorecard that predicts the probability a loan applicant will default, and converts that probability into a 300-900 credit score with a rating band.

1. PROJECT OVERVIEW

Small finance companies and NBFCs lend to a borrower base that often has thin or no credit-bureau history, which makes bureau-score-based underwriting insufficient. This project delivers a probability-of-default scorecard built for that setting.

The model is a binary classifier over 13 features, combining application data with engineered behavioural ratios - loan-to-income, delinquency ratio, and average days-past-due per delinquency. The output is a default probability, rescaled onto a 300-900 credit score with four rating bands so that a credit officer can act on it directly.

Feature selection follows established credit-scoring practice rather than generic machine learning. **VIF** removes multicollinear predictors; **Weight of Evidence and Information Value** select categorical features. WOE linearises a variable against log-odds, which is precisely the scale logistic regression operates on.

Both logistic regression and XGBoost were tuned with Optuna across two class-imbalance strategies. **XGBoost scored marginally higher** - 0.975 against 0.946 on the tuned F1 objective - but logistic regression was selected for production: in regulated credit the coefficients must support adverse-action reasoning, and the accuracy difference did not justify losing that property.

The model is delivered as a Streamlit application that scores an application in real time.

2. THE PIPELINE, AS BUILT

```
Three source tables (customers, loans, bureau_data)
        |
        v
Train/test split  -- 37,500 / 12,497   (done BEFORE cleaning, to avoid leakage)
        |
        v
Cleaning  -- missing values, duplicates, outlier removal on processing_fee,
             correction of data-entry errors in loan_purpose
        |
        v
Feature engineering  -- loan_to_income, delinquency_ratio, avg_dpd_per_delinquency
        |
        v
Feature selection  -- VIF (numeric multicollinearity)
                      WOE / Information Value (categorical predictive power)
        |
        v
Encoding + MinMaxScaler
        |
        v
Model search  -- Logistic Regression vs XGBoost
                 RandomizedSearchCV, then Optuna (TPE)
                 Imbalance: RandomUnderSampler, SMOTETomek,
                            class_weight='balanced', scale_pos_weight
```

```
        |
        v
Evaluation  -- ROC/AUC, Gini, decile rank-ordering, KS
        |
        v
FINAL: LogisticRegression  ->  artifacts/model_data.joblib
```

One detail worth calling out: the train/test split happens *before* cleaning and feature engineering. That ordering is deliberate - fitting imputation, outlier rules or a scaler on the full dataset leaks test information into training and inflates every metric.

3. THE 13 FEATURES

Feature	Type	What it captures
age	Numeric	Applicant age
loan_tenure_months	Numeric	Requested term
number_of_open_accounts	Numeric	Active credit lines
credit_utilization_ratio	Numeric	Outstanding debt / credit limit
loan_to_income	**Engineered**	Loan amount / annual income - affordability
delinquency_ratio	**Engineered**	Past dues / total payments - payment behaviour
avg_dpd_per_delinquency	**Engineered**	Severity of each delinquent event
residence_type_Owned / _Rented	One-hot	Reference category: Mortgage
loan_purpose_Education / _Home / _Personal	One-hot	Reference category: Auto
loan_type_Unsecured	One-hot	Reference category: Secured

The three engineered ratios are the ones to talk about. Raw loan amount and raw income are far weaker than their ratio, because affordability is relative. Same logic for delinquency: two past dues means something very different on a 6-payment history than on a 60-payment history.

4. FEATURE SELECTION - THE CREDIT-SCORING PART

VIF - Variance Inflation Factor

Measures how much a predictor is explained by the *other* predictors. `VIF = 1 / (1 - R2)` from regressing each feature on the rest. Above roughly 5-10 indicates problematic multicollinearity. It doesn't hurt predictive accuracy, but it makes coefficients unstable and sign-flipped - **which matters enormously here, because the whole reason for choosing logistic regression was that the coefficients are interpretable.**

WOE - Weight of Evidence

For each bin of a variable: `WOE = ln(%good / %bad)`. It converts a categorical or binned numeric variable into a single numeric value that is **linear in log-odds** - which is precisely the scale logistic regression operates on. It also handles missing values as their own bin and is robust to outliers.

IV - Information Value

Aggregates WOE across bins into one predictive-power number per feature:

```
IV = SUM over bins of  (%good - %bad) x WOE
```

IV range	Interpretation
< 0.02	Useless - drop
0.02 - 0.1	Weak
0.1 - 0.3	Medium
0.3 - 0.5	Strong
> 0.5	**Suspiciously strong** - often a leakage signal, investigate

The last row matters for this project. See section 7.

5. CLASS IMBALANCE - WHAT WAS TRIED

The test set is 11,423 non-defaults against 1,074 defaults - an **8.6% default rate**. Left untreated, a model can score 91% accuracy by predicting 'no default' every time.

Approach	What it does	Outcome
RandomUnderSampler	Drops majority-class rows until balanced	Fast, but throws away most of the data
SMOTETomek	SMOTE oversampling + Tomek-link cleaning of borderline cases	Used for the final training set
`class_weight='balanced'`	Reweights the loss function instead of resampling	Selected by Optuna for the final logistic model
`scale_pos_weight`	XGBoost's equivalent reweighting	Tuned 1-10 in the XGBoost trials

The point to make: resampling was tested, but the final model relies on `class_weight='balanced'`. Reweighting is generally preferable to synthetic oversampling - it achieves a similar effect with no risk of manufacturing implausible records, and no leakage hazard if the resampling is accidentally applied before the split.

6. EVALUATION - AND WHAT EACH METRIC IS FOR

Metric	Value	What it tells you
ROC-AUC	**0.984**	Ranking quality across all thresholds
Gini	**0.967**	2 x AUC - 1. Same information, credit-industry convention
KS statistic	per decile	Max separation of cumulative good vs bad - speaks to the single best cut-off
Rank-ordering	decile table	Does the bad rate fall monotonically as score rises? A scorecard that doesn't rank-order is unusable regardless of AUC
Accuracy	0.93	Near-meaningless at 8.6% base rate - don't lead with it
Recall (default)	**0.94**	Catches 94% of actual defaults
Precision (default)	**0.56**	But 44% of flagged applications are false positives

The precision-recall trade-off

Recall 0.94 with precision 0.56 is a deliberate operating point, not an accident - it comes from `class_weight='balanced'`. The justification: in origination a missed default costs the loan value, while a false positive costs a manual review. Those are wildly asymmetric, so leaning toward recall is correct. **The threshold should however be set from the business cost ratio, not from F1**, which weights precision and recall equally - and that is almost never what a lender wants.

7. MODEL PERFORMANCE - A CAVEAT ON THE AUC

AN AUC OF 0.984 IS IMPLAUSIBLY HIGH FOR CREDIT DEFAULT

Typical credit scorecards land at **0.75-0.85**. At 0.98 the model is separating defaulters almost perfectly, which in this domain is far more likely to indicate **target leakage** than exceptional modelling. The headline figure should not be treated as a validated result until this is resolved.

The most likely cause

Two of the engineered features - `delinquency_ratio` and `avg_dpd_per_delinquency` - are derived from repayment history. **If that history spans or follows the default event, those features partly encode the target.** A borrower who defaulted will mechanically show a high delinquency ratio, because defaulting *is* a sequence of missed payments.

In credit modelling this is the canonical leakage trap: behavioural features must be computed from a strictly pre-application observation window. If the observation window overlaps the performance window, the model is partly predicting the past.

Corrective action

Recompute `delinquency_ratio` and `avg_dpd_per_delinquency` from a strictly pre-application observation window, then re-baseline the model. The AUC should be expected to fall substantially, and the lower figure would be the trustworthy one. Until that work is done, the 0.984 should be reported with the caveat attached rather than presented as a performance result.

8. FROM PROBABILITY TO CREDIT SCORE

```
credit_score = 300 + (1 - default_probability) x 600


300-499  Poor        500-649  Average
650-749  Good        750-900  Excellent
```

Know the limitation. This mapping is linear in *probability*. Industry scorecards are linear in *log-odds*:

```
score = Offset + Factor x ln(odds)      where  Factor = PDO / ln(2)
```

PDO is "points to double the odds", typically 20. That construction means a fixed number of points always represents the same change in odds, anywhere on the scale - which is what makes FICO-style scores comparable. Linear-in-probability compresses discrimination at the extremes.

Second limitation: the band cut-offs (500/650/750) were set in the app, not derived. In practice they'd come from swap-set analysis - plotting approval rate against expected bad rate and choosing where the marginal approved applicant breaks even.

9. WHY LOGISTIC REGRESSION, WHEN XGBOOST SCORED HIGHER

	Logistic Regression	XGBoost
Tuned F1 (Optuna)	0.946	**0.975**
Interpretability	**Direct coefficients**	SHAP approximation only
Adverse-action reasoning	**Straightforward**	Difficult to defend
Regulatory precedent	**Industry standard**	Challenger model
Monotonicity	**Guaranteed by sign**	Needs explicit constraints
Selected	**YES**	No

XGBoost won on raw F1, but logistic regression was selected for production. In lending there is a legal obligation to give a specific reason for a decline, and a model whose coefficients can be read directly makes that straightforward. A three-point F1 difference does not justify losing it. The appropriate framing is XGBoost as a challenger model and for feature discovery, with the interpretable model in production - revisited if the challenger ever wins *materially*.

10. WHAT I WOULD DO DIFFERENTLY

- **Investigate the leakage first.** Recompute behavioural features from a strictly pre-application window and re-baseline. Everything else is secondary to this.
- **Wrap preprocessing in a Pipeline.** The scaler was fitted on 18 columns while the model uses 13, so the app injects 10 placeholder values just to satisfy `scaler.transform`. A `ColumnTransformer` inside a `Pipeline` serialised as one object removes the training-serving skew risk.
- **Move the scorecard into the notebook** and derive it from log-odds, rather than defining the scaling and the band cut-offs in the app with no validation behind them.

- **Add calibration assessment** - a reliability curve and Brier score. A probability that drives a decision needs to be right in level, not just in rank order.
- **Add reject inference.** The model only learns from applicants who were approved, so it is trained on a censored population and is biased about those previously declined.
- **Set the threshold from business costs** rather than F1, and document the chosen operating point and its expected false-positive volume.